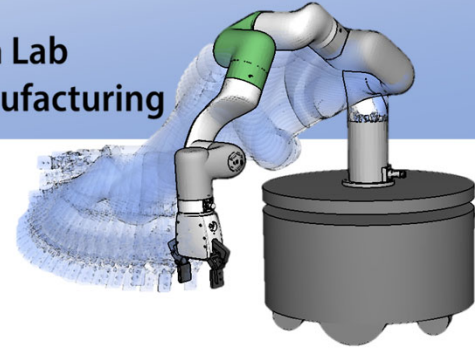


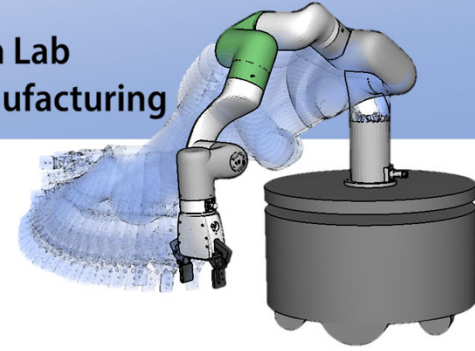


コンピュータ基礎

原田研究室 准教授 万 偉偉

基礎工学部 F522号室

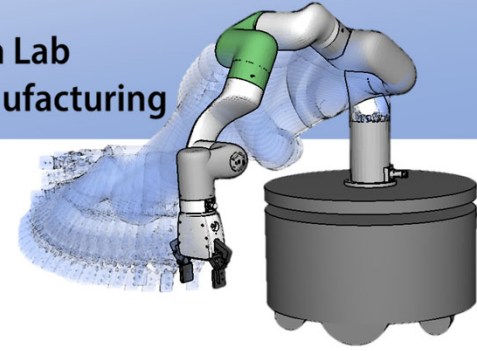
wan@sys.es.osaka-u.ac.jp



情報処理演習では

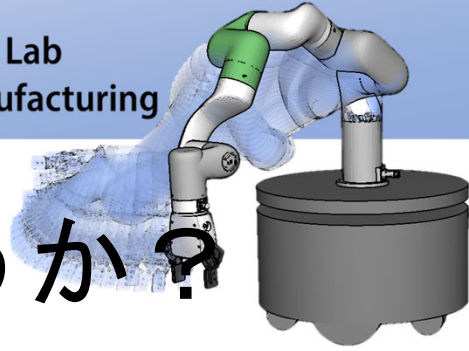
- 変数
- 制御構造：条件分岐と繰り返し
- 関数
- 配列
- 配列を引数とした関数
- 文字配列
- ポインタ
- ファイル

C言語の基礎中の基礎



コンピュータ基礎は

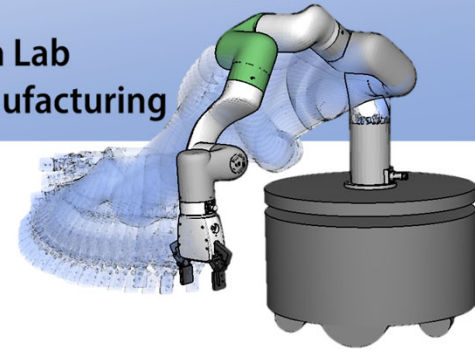
- 情報科学(コンピュータ科学)の分野の授業である
- 当コースの全ての実験、各研究室での研究に必要なスキルである
- 社会に出たときに必要とされる可能性が極めて高い



コンピュータ基礎では何を習うか？

- ポインタ
- 構造体
- データ構造とアルゴリズム
- 言語記法、プログラム記法

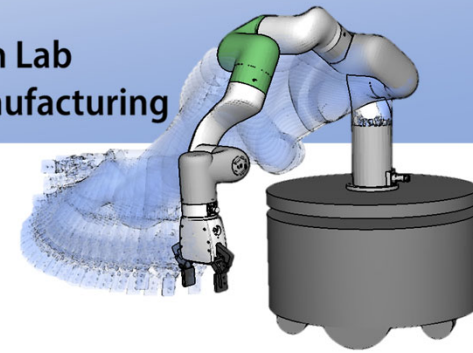
例題や演習ではC言語を用いるが、言語に依存しない説明や、他の言語のことも学ぶ



授業の方針

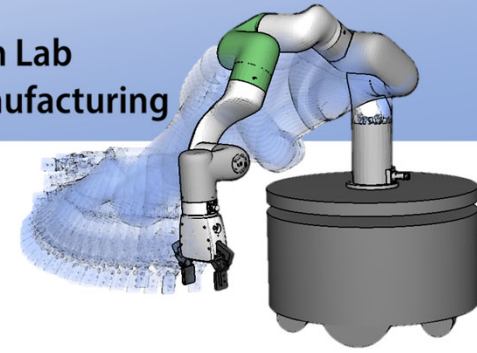
初めての専門科目の
授業である

知能システム学コース専門の授業



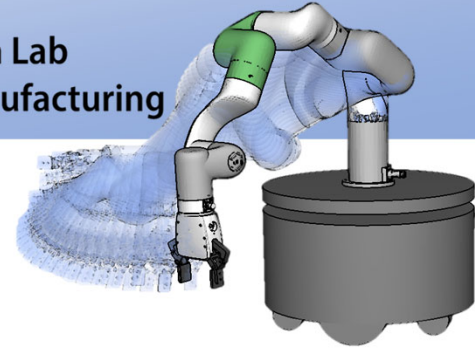
授業の方針

- コンピュータ言語は**教えられるものではない**。自ら学ぶものである
- 授業では、完全には説明しきれない
- **全員教科書を購入**して、**各自で読む**ように
- 学んだ内容は、実践しない限りものにならない
- 演習は自力でこなすように
- 授業も演習も、各自の習得の手助け（きっかけ）であって、100%教えてくれると思わないように
- 資料は事前にダウンロードして各自で印刷
- 担任の先生：1～7回 万；8回～14回 中村



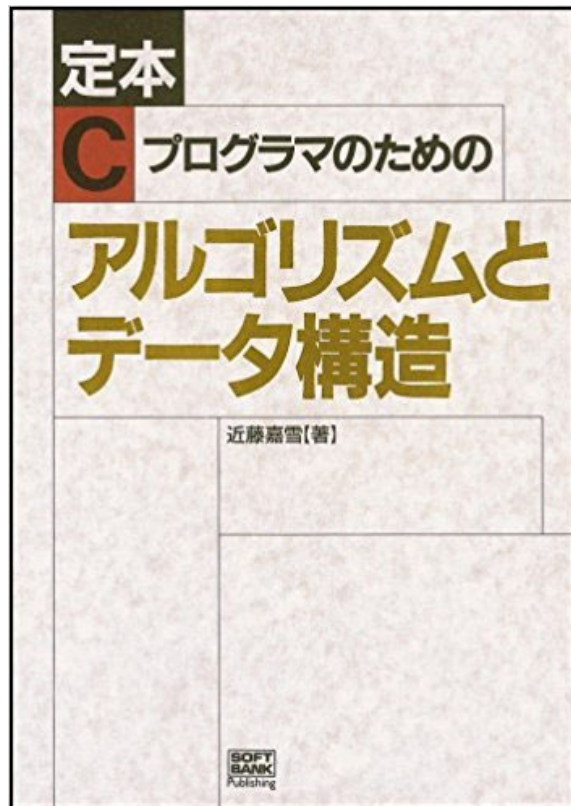
授業の方針

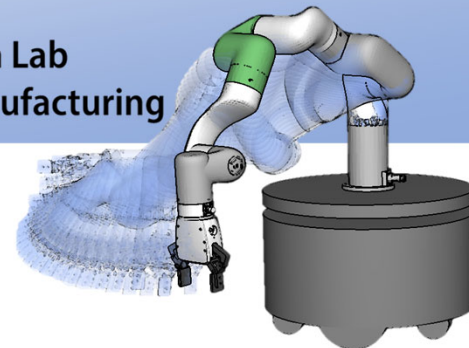
- 自然言語のAI Codingがあっても本授業の基礎知識が必要
 - 生成したコードの理解
 - 専門用語やアルゴリズムの基本
 - 実例



教科書

- 「定本 Cプログラマのためのアルゴリズムとデータ構造」、ソフトバンククリエイティブ社



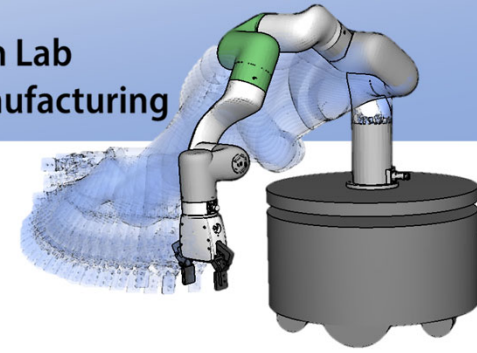


導入：アルゴリズムと データ構造

原田研究室 准教授 万 偉偉

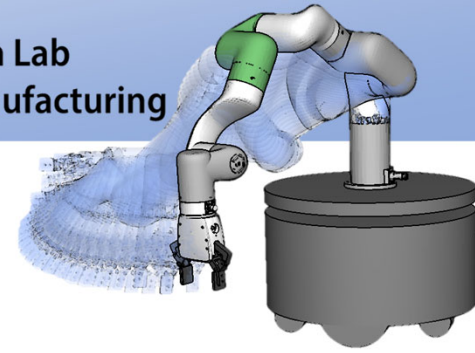
基礎工学部 F522号室

wan@sys.es.osaka-u.ac.jp



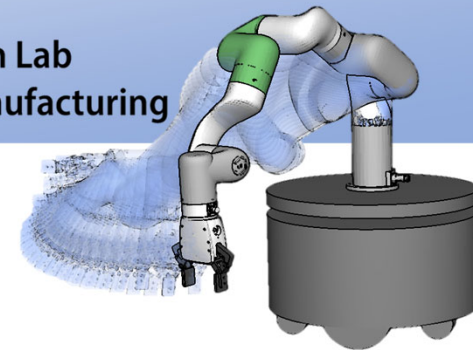
本日の講義で習う内容

- アルゴリズムとは
- プログラミングとは
- データ型
- データ構造
- 演算子



アルゴリズムの設計と解析

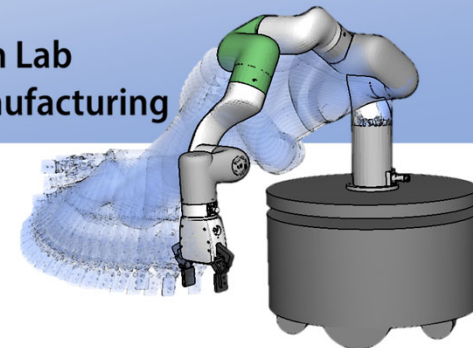
- アルゴリズムとは
 - 問題を解くための論理または手順
- プログラム
 - データ構造を利用して, アルゴリズムをプログラミング言語で表現したもの



プログラミングの流れ

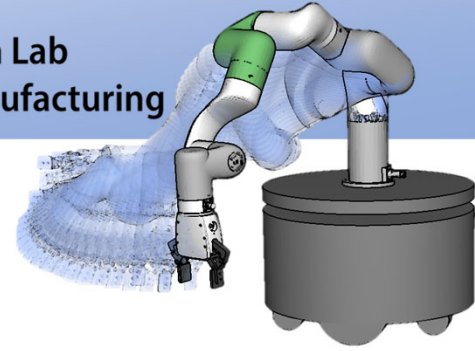
- 問題分析
- 何をするプログラムを書くのか決める
(プログラム仕様の決定)
- どんな**アルゴリズム**, **データ構造**を使用すれば
よいかを選択
- 実際にプログラムを書く
- プログラムが正しく動くことを保証する
(検証)

この講義では,
適切な**アルゴリズム**, **データ構造**を
選ぶ能力を身に付ける



良いアルゴリズムの条件

- 信頼性が高い（精度の良い，正しい結果が得られる）
- 処理効率が良い（計算回数が少ない，処理速度が速い）
- 一般性がある（特定の問題だけでなく，他の問題にも適用できる）
- 拡張しやすい（仕様変更に対し簡単に修正が行える）
- わかりやすい（誰が見ても理解できる．プログラムの保守がしやすい）
- 移植しやすい（他のシステムにも少ない労働で稼働させることができる）



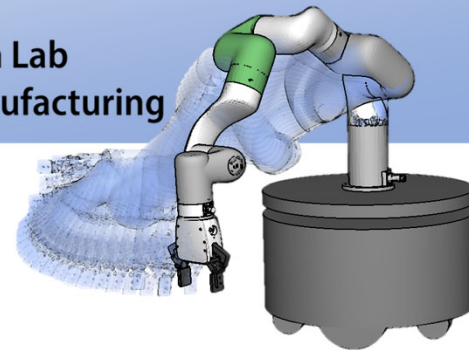
プログラミングの流れ（詳細）

- 問題分析

- 与えられた問題の意味を正確に理解
 - 目的
 - 与えられるデータは何か
 - 求めるべき結果は何か
 - 要求される精度は
 - 処理に与えられた計算量は

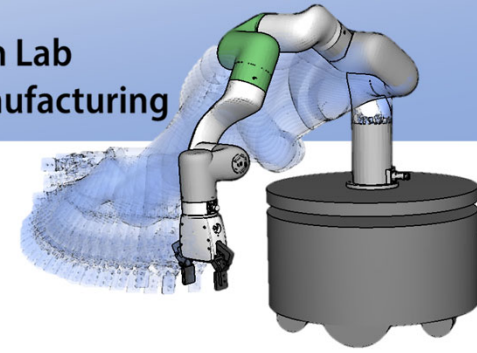
- 計算方法の選択

- どの方法が与えられた問題の処理に適しているか
（アルゴリズム＋データ構造）



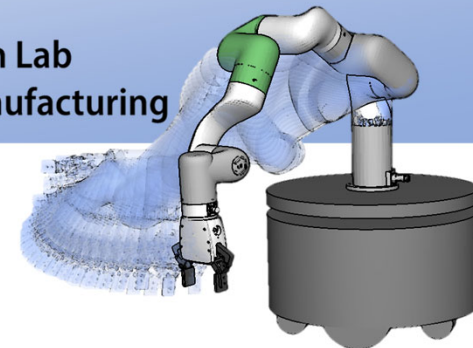
プログラミングの流れ（詳細）

- コーディング
 - 文法的な誤り→言語処理プログラムによりこの時点で発見
- プログラムの検査
 - 論理的な誤り→事前条件，事後条件の検査
 - 中間処理の例外検出・不変条件のチェック
- プログラムの実行

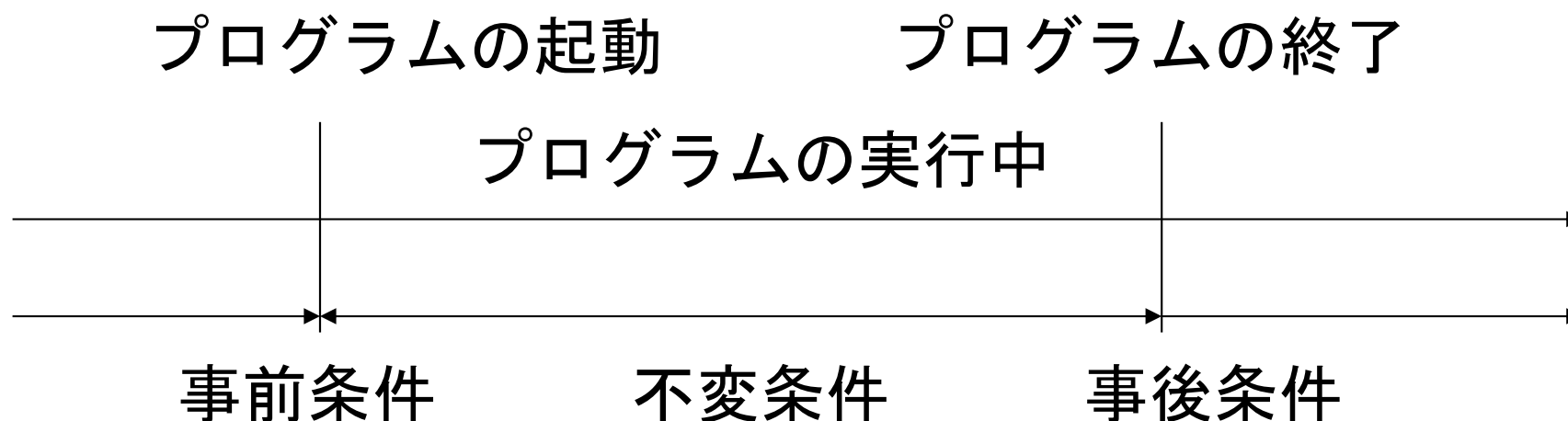


事前条件と事後条件（１）

- 事前条件 (precondition)
 - プログラムが正しく動作するための前提条件
- 事後条件 (postcondition)
 - プログラムが動作した後に成立する条件
- 不変条件
 - プログラムが動作中に成立している条件

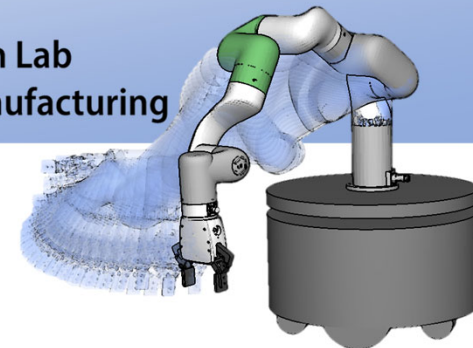


事前条件と事後条件（２）



正しいプログラムとは：

事前条件が成立しているときに起動された場合に、
実行中に不変条件が常に成立しており、かつ、
終了後に事後条件が成立するプログラム



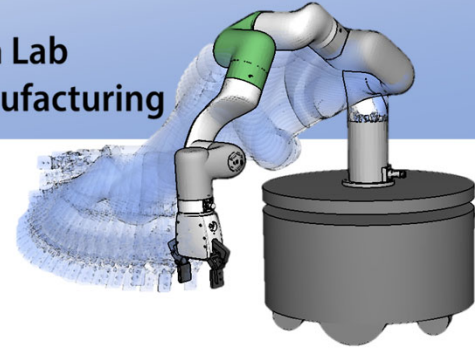
例題 1) 漸化式

n 個の中から r 個を選ぶ組み合わせの数 ${}_nC_r$ を求める

a, b, c の中から 2 個選ぶ組み合わせは,
 ab, ac, bc の 3 通り

$$\text{一般には, } {}_nC_r = \frac{n!}{r!(n-r)!}$$

計算機でこの式を実行した場合, 大きい n の値に対し
 $n!$ がオーバーフローする危険性がある.



例題 1) 漸化式

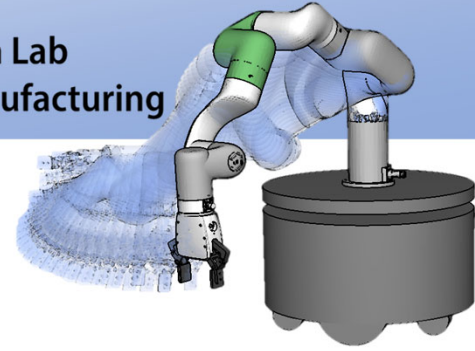
例えば,

$${}_{15}C_5 = \frac{15!}{5!(15-5)!} = \frac{130767436800}{120 \times 3628800} = 3003$$

最終結果はオーバーフローしないが, 分子では???

unsigned int 型が16ビットの場合 → 65535

int 型が16ビットの場合 → -32767-32768

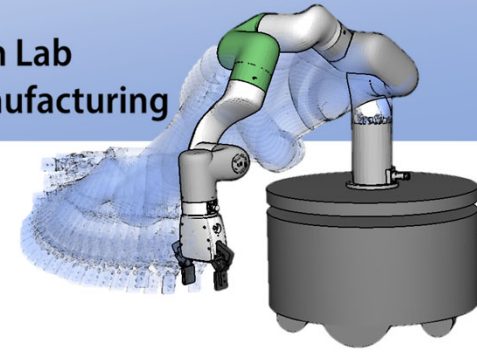


問題を分析する

- 組み合わせは漸化式で表現できる

$${}_nC_r = \frac{n!}{r!(n-r)!} \quad \begin{cases} {}nC_r = \frac{n-r+1}{r} {}nC_{r-1} \\ {}nC_0 = 1 \end{cases}$$

自分自身 $\{{}_nC_r\}$ を次数の低い自分自身 $\{{}_nC_{r-1}\}$ で再帰的に定義



問題を分析する (2)

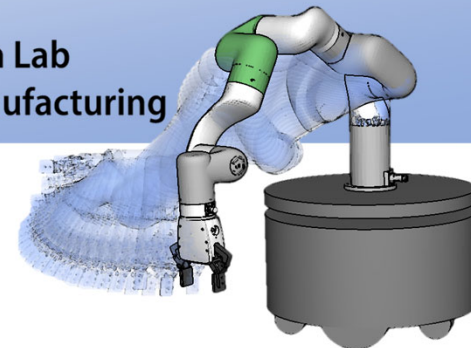
漸化式の表現方法

1) 繰り返しの利用

- ✓ 関数呼び出しのオーバーヘッドが小さい
- x 複雑な再帰的定義を展開する場合,
プログラムが複雑になる

2) 再帰呼び出しの利用

- ✓ 定義通りにプログラムを組みやすく保守しやすい
- x メモリの使用効率が悪い



問題を分析する (3)

漸化式を繰り返して表現する場合：

方針) ${}_nC_0$ を初期値とし, それに係数 $\frac{n-r+1}{r}$ を
順次掛け合わせていく.

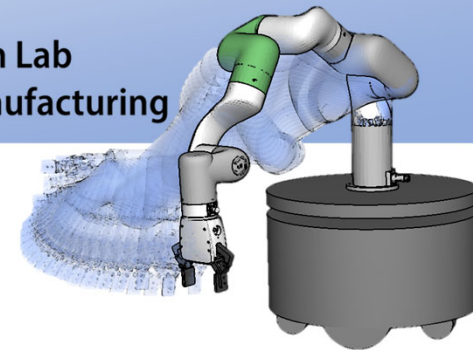
繰り返しの制御には r を利用する.

r を1から始めて, 繰り返し毎に $+1$ を加え,
指定した値になれば停止する.

$$\frac{n!}{r!(n-r)!} = \frac{n!}{(r-1)!(n-(r-1))!} \cdot \frac{1}{r} \cdot (n-r+1)$$

$$\frac{{}_nC_r}{{}_nC_0} = \frac{1}{1} \times \frac{n-1+1}{1} \times \frac{n-2+1}{2} \times \dots \times \frac{n-r+1}{r}$$

${}_nC_1$



問題を分析する (4)

事前条件

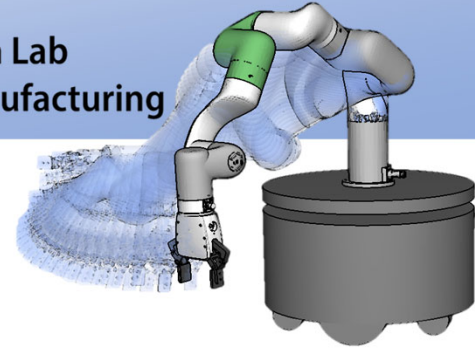
今までの議論が成立するために暗黙的に仮定していたこと
例えば, こういう組み合わせの値は?

$${}_2C_3? \quad {}_3C_{-1}?$$

プログラムが正しく動作するためには

$$n \geq r \geq 0$$

があらかじめ成立していなければならない



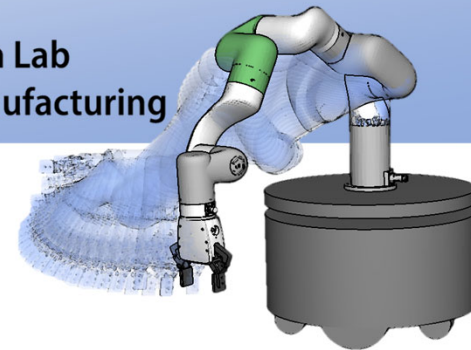
問題を分析する (5)

契約の概念

事前条件を満たした状態でプログラムが呼び出された場合、プログラムは事後条件を満たすことを保証する。

この場合の事後条件とは？

${}_nC_r$ が正しく計算されること

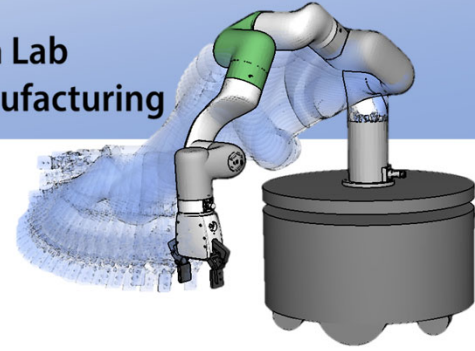


繰り返しによる実装例

```
int combination(int n, int r) {
    int i, p;
    p = 1;
    for (i = 1; i <= r; i++) {
        p *= (n - i + 1);
        p /= i;
    }
    return p;
}
```

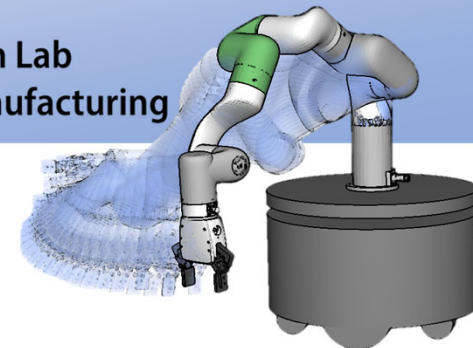
$$\frac{{}_nC_r}{{}_nC_0} = \frac{1}{1} \times \frac{n-1+1}{1} \times \frac{n-2+1}{2} \times \dots \times \frac{n-r+1}{r}$$

${}_nC_1$

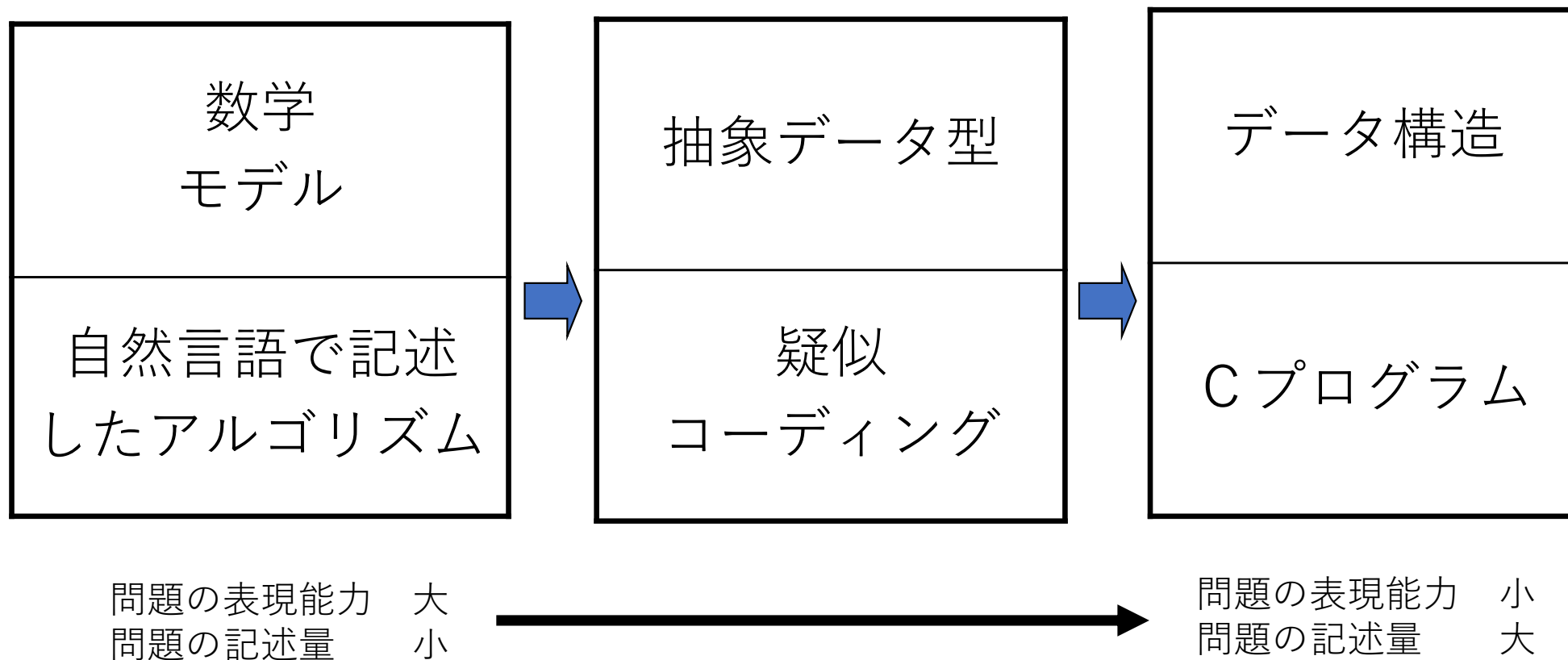


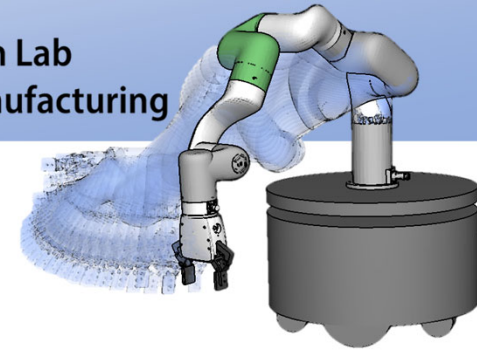
計算方法の選択方法, 考え方

- 疑似コーディング（疑似プログラム）
 - どのようなアルゴリズムを使えば良いか
 - ごく大ざっぱなスケッチ, 全体像を掴む
 - 始めは自然言語で
 - 徐々に事前条件, 事後条件を明らかにしつつ, プログラミング言語に置き換え→段階的詳細化



問題解決の過程

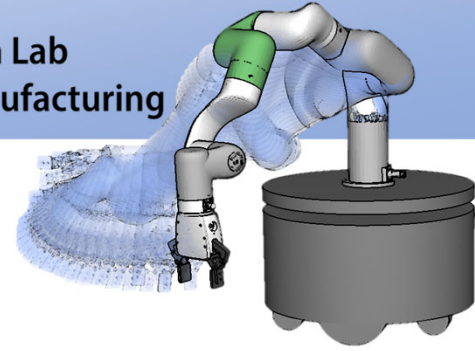




疑似コーディング

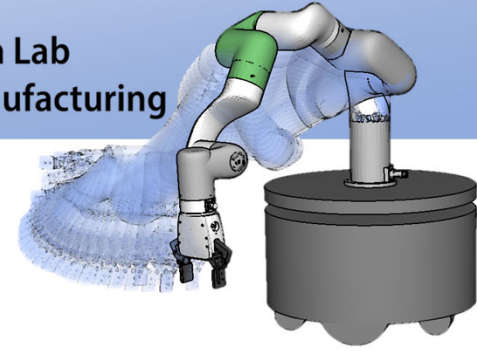
- ・プログラミング言語と自然言語とを利用して、設計の補助として作成するコード

日本語	疑似コーディング	C 言語
X に Y を代入	$X \leftarrow Y$	$X = Y;$
X と Y は等価	$X = Y$	$(X == Y);$
X と Y は等しくない	$X \neq Y$	$(X != Y);$
N 回繰り返し	for j=1 to 50 do endfor	for (i=1;i<=N; ++i)
X が Y より大きいときに実行する	if X>Y then endif	if(X > Y){ }



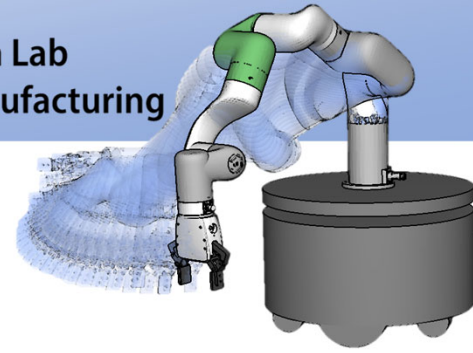
本日の講義で習う内容

- アルゴリズムとは
- プログラミングとは
- データ型
- データ構造
- 演算子



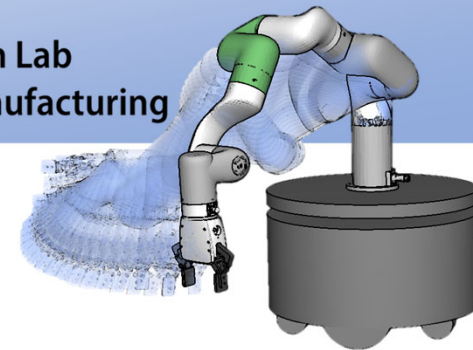
データ型とデータ構造

- データ型 (type of data)
 - ある特定言語のデータの型
 - 例えば, int, char など
- データ構造 (data structure)
 - ある数学モデルを実装するためのデータの集まり
 - いろいろなデータ型をもつデータの集まり



C言語のデータ型

- 基本型
 - 文字型, 整数型, 実数型
- 構造型
 - 配列, 構造体, 共用体
- ポインタ型



データ構造

- ある数学モデルを実装するための様々なデータ型を持つデータの集まり

- リスト(list)
- 待ち行列(queue)
- スタック(stack)
- 木(tree)
- ハッシュ表(hash table)
- グラフ(graph)

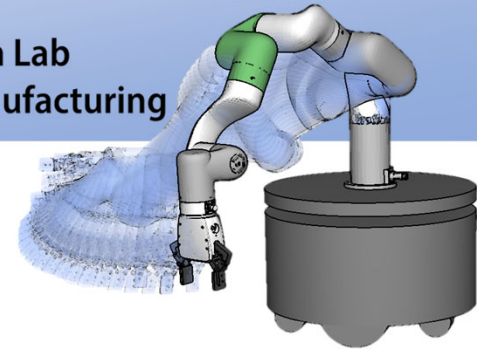
抽象データ型

(Abstract Data Type)

=データ構造 + 取り扱う手法 (手続き)

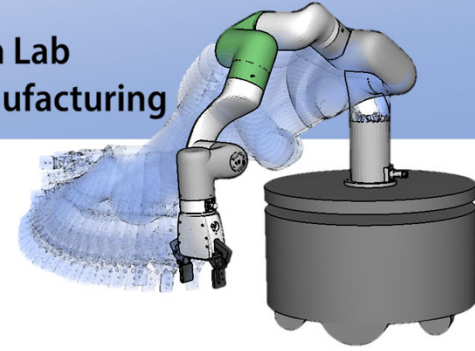
これらのデータ構造は、配列型のデータ領域と格納場所を示す整数型カウンタで実装できる。

もちろん、ポインタと構造体を使っても実装できる。



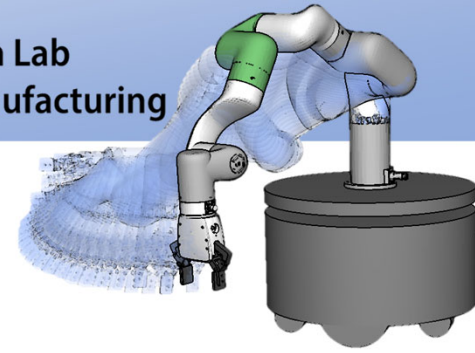
抽象データ型(ADT)

- データのカプセル化手段
- データ表現と手続きの分離
- 内部データの一貫性保守
- クラス(Class)ともいわれる
 - SIMULA67がALGOL60をベースに開発された最初のクラスを持つ言語
 - クラスを持つ言語 Objective-C, C++, C#, Java, Python など



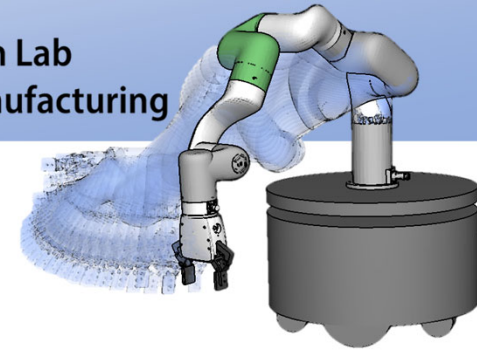
抽象データ型の図式表現

Robot
Linklength: Double Jointangle: Double
Double Getjoint() Bool Moveto()



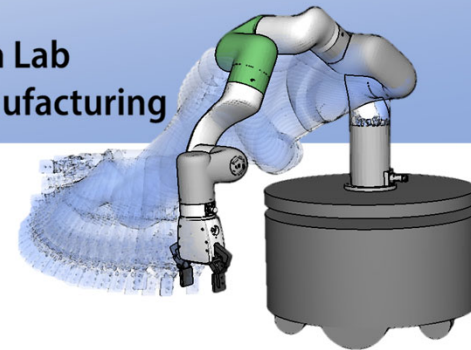
型の表記法

- 型名(例 : int, float, robot など)
- 関数型
 - 型名 : 入力リスト型 $\rightarrow$ 出力リスト型
 - $F: \text{int}, \text{int} \rightarrow \text{int}, \text{int}$
 - 2つの整数型を入力とし, 2つの整数型を出力とする型 F
- $\text{int}^* F(\text{int } a, \text{int } b)$
- $\text{robot } F(\text{xx}, \text{xxx})$



配列と構造体（C言語）

- 配列(array)->抽象データ型
 - データ構造：同じデータ型を持った要素を集めたもの
 - 手続き：配列の要素は，添字（インデックス，index）により参照可能
- 構造体(structure)
 - 異なるデータ型を意味的にひとまとまりにしたもの



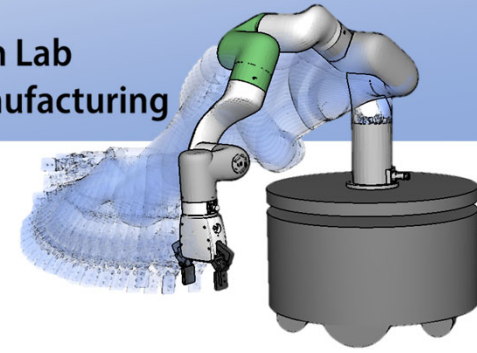
配列（C言語）

- データ型 配列名 [要素数] ;
 - 指定されたデータ型が要素数分だけ連続した格納領域の宣言

例) `int data[4];`

0～3番までの4個の `int` 領域が確保される.

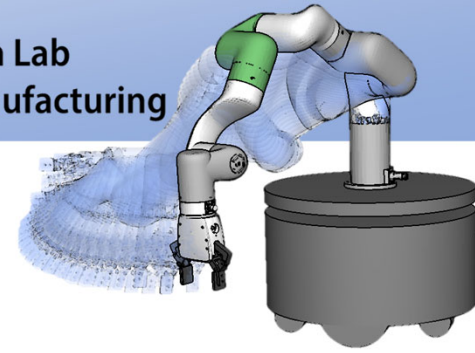
<code>data[0]</code>	<code>data[1]</code>	<code>data[2]</code>	<code>data[3]</code>
<code>int</code>	<code>int</code>	<code>int</code>	<code>int</code>



配列の初期化

- データ型 配列名[要素数] = {定数, 定数, ..., 定数}

例) `int data[4] = { 7, 4, 3, 10 };`



例題 2

- 例えば，最大値を求めるには
 - 最大値の条件：
 - 他のすべてのデータより大きい（事後条件）

$$\exists m \forall x [m \geq x]$$

- すべてのデータと比較が必要

疑似言語記述：

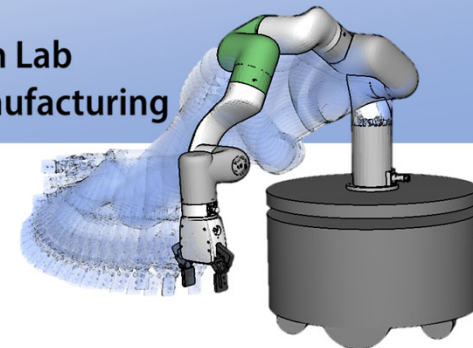
```
1) m ← data[0]
2) for (i ← 1 .. N-1)
    if m < data[i] then m ← data[i]
```

繰り返す

条件分岐

最大値の条件

最大値になるよう修正



配列データの最大値計算

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int data[10] = { 9, 8, 7, 6, 5, 4, 3, 2, 1, 0 };
```

```
    int i, m, n = 10;
```

配列の要素数

```
        m = data[0];
```

```
        for (i=1; i<n; i ++)
```

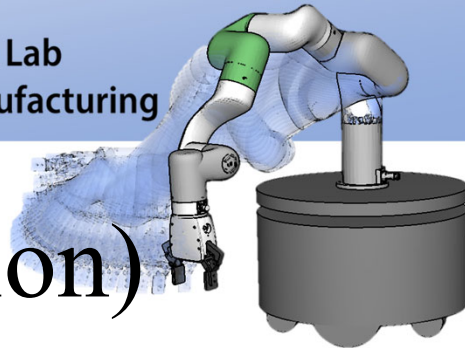
```
            if (m < data[i]) m = data[i];
```

```
        printf( "maximum = %d\n", m );
```

```
        return 0;
```

```
    }
```

疑似言語をC言語に変換
した部分

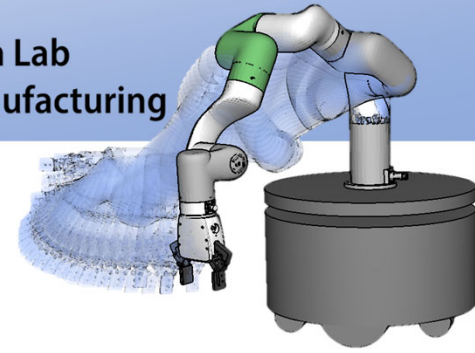


定義(Definition)と宣言(Declaration)

- 宣言(Declaration)
 - 変数や関数などをこれから利用することを明示すること
- 定義(Definition)
 - 変数や関数などの中身を決定的すること
 - 定義すると計算機の中にそのための記憶領域が確保される

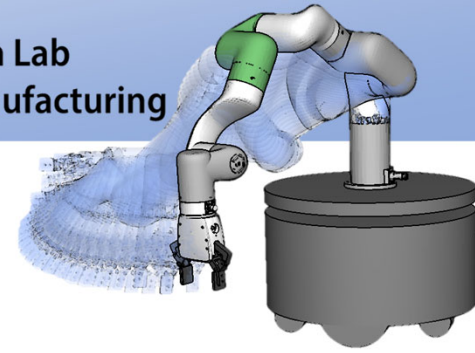
C言語では→extern 宣言以外は
宣言すると同時に定義したことになる

構造体 → 宣言・定義
メモリが確保できる 同時に定義
後で動的に配る場合 分ける



本日の講義で習う内容

- アルゴリズムとは
- プログラミングとは
- データ型
- データ構造→抽象データ型
 - 型
- 演算子



演算子

- オペランドoperand : 式中的変数や定数

- 演算子の種類

- 単項演算子

例) $-a$

- 2項演算子

例) $a+b$

- 3項演算子

例) $(a > 0) ? a : -a$

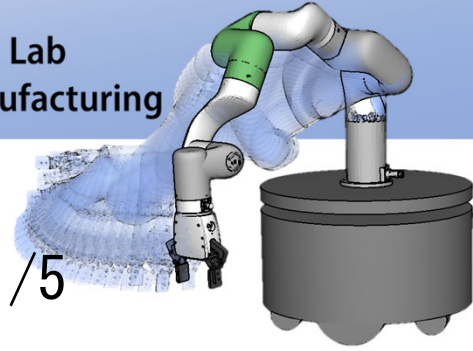
- 演算子の例

- 算術演算子

例) $a+b$

- インクリメント演算子

例) $++a$



演算子：優先順位と結合規則 1/5

- 例

$y = a + b * c / d;$

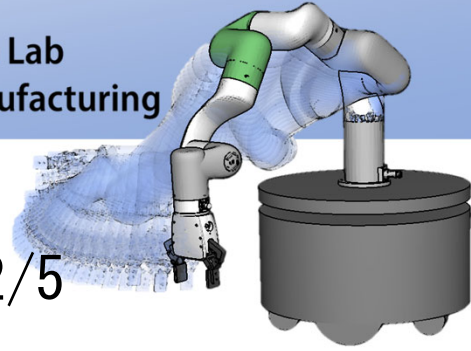
$y=a$ よりも $a+b$ よりも $b*c$ が優先される

代入演算子 $=$ よりも、加減演算子 $+$ よりも、乗除演算子 $*$ の方が優先される

$y = a + b * c / d;$

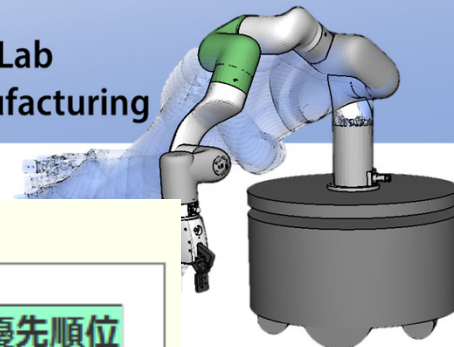
$/d$ は $b*c$ が実行されてから実行される

乗除演算子である $*$ と $/$ は、優先順位は同じだが、左にあるものから結合し計算される



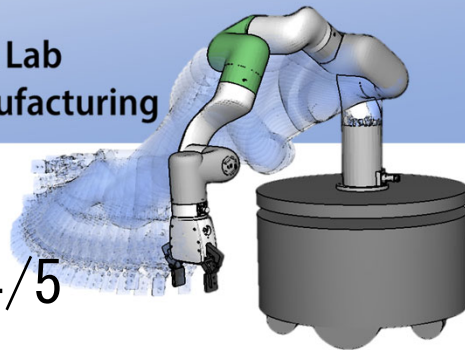
演算子：優先順位と結合規則 2/5

- 優先順位：式中のどの演算子から先に演算を行うか
- 結合規則：式中で同一の優先順位を持つ演算子が現れた場合に、左右どちらから先に演算を行うか



【演算子の優先順位と結合規則】

種類	演算子	結合規則	優先順位
関数, 添字, 構造体, 後置増分減分	() [] . -> ++ --	左=>右	高 ↑
前置増分減分, 単項式	++ -- ! ~ + - * & sizeof	左<=右	
キャスト	(型)		
乗除余	* / %	左=>右	
加減	+ -		
シフト	<< >>		
比較	< <= > >=		
等値	== !=		
ビットAND	&		
ビットXOR	^		
ビットOR			
論理AND	&&		
論理OR			
条件	?:	左<=右	
代入	= += -= *= /= %= &= ^= = <<= >>=		
コンマ	,	左=>右	低



演算子：優先順位と結合規則 4/5

- ややこしいけど，非常によく使う式

$y = *p++;$

種類	演算子	結合規則	優先順位
関数, 添字, 構造体, 後置増分減分	() [] . -> ++ --	左=>右	高
前置増分減分, 単項式	++ -- ! ~ + - * & sizeof	左<=右	↑
キャスト	(型)		↓

この表に従うと，ポインタpのアドレスが，式の実行後にインクリメント(++)される．インクリメントされる前のアドレスで *pで値を取得する．その値を = により，yに代入する．



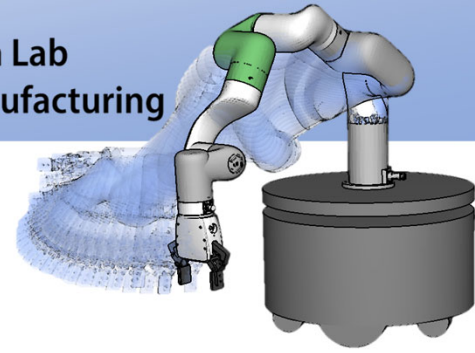
種類	演算子	結合規則	優先順位
関数, 添字, 構造体, 後置増分減分	() [] . -> ++ --	左=>右	高
前置増分減分, 単項式	++ -- ! ~ + - * & sizeof	左<=右	↑
キャスト	(型)		↓

! と ++（前置増減演算子）が同じ優先順位

$y = !++a$

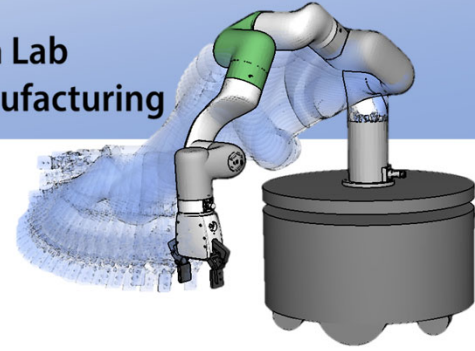
1. まず **a** がインクリメントされる
2. インクリメントされた値を否定する
3. 上記を **y** に代入する

※ただし，同じ優先順位で右→左の結合規則を持つような演算を行うことは，ほとんどない。



本日の講義で習った内容

- アルゴリズムとは
- プログラミングとは
- データ型
- データ構造
- 演算子



宿題

- 来週の授業が開始する前（午前9時締切），CLEで提出ください。
- AIにお任せは身に付けられません。落ち着いて自ら解くこと。
- 解いてからAIに確認してもらう。
- 提出内容にAIからもらった心得を書くこと。

→ 心得はないと不合格